Theory of Computation

沈威宇

August 5, 2026

Contents

1 Theory of Computation (TOC)

I Formal Language Theory

i Alphabet

An alphabet Σ of a formal language is a finite set of symbols, called characters or symbols.

ii String or word

Strings (or words) over alphabet Σ are defined as follows:

- The empty sequence is a string, called empty string and denoted by ε.
- A single character $a \in \Sigma$ is a string.
- A concatenation of strings is a string, denoted by writing them side by side.

The length of a string, denoted by $|\cdot|$, is defined as follows:

- The empty string ε has length 0.
- A single character $a \in \Sigma$ has length 1.
- A string of length m concatenated with a string of length n yields a string of length $m + n$.

iii Kleene star, Kleene operator, or Kleene closure of an alphabet

Given an alphabet V, define

$$V^0 = \{\varepsilon\},$$

and define recursively for $i \in \mathbb{N}$:

$$V^i = \{wv \mid w \in V^{i-1} \wedge v \in V\}.$$

The Kleene star on V is defined as

$$V^* = \bigcup_{i \in \mathbb{N}_0} V^i.$$

The Kleene plus on V is defined as

$$V^+ = \bigcup_{i \in \mathbb{N}} V^i.$$

iv Formal language

A formal language, or, when the context is clear, language, L over Σ is any subset of Σ^*. A word $w \in \Sigma^*$ is called well-formed iff $w \in L$.

v Kleene star, Kleene operator, or Kleene closure of a language

Given a language V, define

$$V^0 = \{\varepsilon\},$$

and define recursively for $i \in \mathbb{N}$:

$$V^i = \{wv \mid w \in V^{i-1} \wedge v \in V\}.$$

The Kleene star on V is defined as

$$V^* = \bigcup_{i \in \mathbb{N}_0} V^i.$$

The Kleene plus on V is defined as

$$V^+ = \bigcup_{i \in \mathbb{N}} V^i.$$

vi Concatenation of languages

Given two formal languages L_0, L_1 over the same alphabet, the concatenation of them, denoted by L_0L_1 or $L_0 \cdot L_1$, is defined as

$$L_0L_1 = \{xy \mid x \in L_0 \wedge y \in L_1\}.$$

vii Reflexive and transitive closure of vdash

$\vdash^*$ is the reflexive and transitive closure of the binary relation $\vdash$, that is, if

$$C_0 \vdash C_1 \vdash \ldots \vdash C_k,$$

then

$$C_0 \vdash^* C_k.$$

viii (Unrestricted or type 0) (formal) grammar

A grammar can be defined as a four-tuple

$$G = (V, \Sigma, R, S)$$

where

- V is a set of nonterminal symbols, abstract symbols that represent syntactic categories.
- Σ is a set of terminal symbols, the actual characters that can appear in a string, i.e., alphabet, with $V \cap \Sigma = \varnothing$.
- $R \subseteq (V \cup \Sigma)^* V (V \cup \Sigma)^* \times (V \cup \Sigma)^*$ is a set of production rules (also called rewrite rules), which are pairs of a nonempty string of nonterminal symbols and terminal symbols and a string of nonterminal symbols and terminal symbols, in which a rule is denoted as

  $$A \to a$$

  for $(A, a) \in R$ and called A rewrites to a. For any $v, w \in (V \cup \Sigma)^*$ and $A \to a$, we write $vAw \vdash vaw$.
- $S \in V$ is a start symbol.

The language defined by a grammar G, i.e., all $w \in \Sigma^*$ such that $S \vdash^* w$, is denoted as $L(G)$.

ix Context-free grammar (CFG) or type 2 grammar

A context-free grammar is a grammar of which the production rules are pairs of a nonterminal symbol and a string of nonterminal symbols and terminal symbols, i.e.,

$$R \subseteq V \times (V \cup \Sigma)^*.$$

x Context-free language (CFL) or type 2 language

A context-free language is a language defined by a context-free grammar.

xi Deterministic context-free language (DCFL)

A language is a deterministic context-free language iff there exists a deterministic pushdown automaton (DFPA) that recognize it.

xii Regular grammar or type 3 grammar

A regular grammar is a grammar of which either all the production rules are right-linear or all the production rules are left-linear.

A production rule is right-linear if it is in the set

$$V \times \Sigma \cup V\Sigma^* \cup \{\varepsilon\}.$$

A production rule is left-linear if it is in the set

$$V \times \Sigma \cup \Sigma^* V \cup \{\varepsilon\}.$$

xiii Regular language or type 3 language

A regular language is a language defined by a regular grammar.

The collection of regular languages over an alphabet Σ can be defined recursively with:

- The empty language $\varnothing$ is a regular language.
- For each $a \in \Sigma$, the singleton $\{a\}$ is a regular language.
- If A is a regular language, then A^* is a regular language.
- If A and B are regular languages, then $A \cup B$ and $A \cdot B$ are regular languages.
- No other languages over Σ are regular.

xiv Context-sensitive grammar (CSG) or type 1 grammar

A context-sensitive grammar is a grammar such that all production rules are either $S \to \varepsilon$ or in the form

$$vAw \to vaw$$

where $A \in V$, $v, w \in (V \cup \Sigma \setminus \{S\})^*$, and $a \in (V \cup \Sigma \setminus \{S\})^+$.

xv Context-sensitive language (CSL) or type 1 language

A context-sensitive language is a language defined by a context-sensitive grammar.

xvi Recursively enumerable (RE), recognizable, partially decidable, semidecidable, Turing-acceptable, Turing-recognizable, or type 0 language

A recursively enumerable language is a language defined by an unrestricted grammar.

II Automata Theory

i Automata

An automaton (plural: automata) can be defined as a four-tuple

$$M = (\Sigma, S, s_0, \delta),$$

where

- Σ is an input alphabet.
- S is a set of all states.
- $s_0 \in S$ is the starting state (also called start state).
- $\delta \subseteq S \times \Sigma \times S$ is a transition function (also called next-state function), which are triples of current state, input character, and next state, in which each element is denoted as

  $$(s, \sigma) \to t \quad \text{or} \quad \delta(s, \sigma) = t$$

  for $(s, \sigma, t) \in \delta$.

  A general transition function and a general automaton is called nondeterministic. If δ satisfies the constraint that

  $$\forall (s, \sigma) \in S \times \Sigma : (s, \sigma, t) \in \delta \land (s, \sigma, u) \in \delta \implies t = u,$$

  the transition function and the automaton is called deterministic. A deterministic transition function can also be written as a partial function

  $$\delta : S \times \Sigma \to S$$

  mapping pairs of current state and input character to next states.

Given an input word $w = a_1 a_2 \dots a_n \in \Sigma^*$, for the ith step of the computation (also called the ith transition), for all $(s_{i-1}, a_i, s_i) \in \delta$, the automaton may transition from s_{i-1} to s_i, denoted as $s_{i-1} \vdash s_i$ or $(s_{i-1}, w_i) \vdash (s_i, w_{i+1})$, where $w_i = a_i a_{i+1} \dots a_n$ and $w_{n+1} = \varepsilon$. s_n is called the final state.

ii Extended transition function

The transition function $\delta \subseteq S \times \Sigma \times S$ can be extended inductively into $\bar{\delta} \subseteq S \times \Sigma^* \times S$, which are triples of current state, remaining input word, and final state, with, for the empty string ε,

$$\{(s, \varepsilon, t) \in \bar{\delta}\} = \{(s, \varepsilon, s) \mid s \in S\},$$

and for any fixed string $wa \in \Sigma^*$ where $a \in \Sigma$ is the last symbol and $w \in \Sigma^*$ is the (possibly empty) rest of the string,

$$\forall t \in S : (s, wa, t) \in \bar{\delta} \iff \exists (s, w, q) \in \bar{\delta} \text{ s.t. } (q, a, t) \in \delta.$$

iii Acceptor or recognizer

An acceptor (also called recognizer) is an automaton with a set of accepted (final) state and can be defined as a five-tuple

$$M = (\Sigma, S, s_0, \delta, F),$$

where

- Σ is an input alphabet.
- S is a set of all states.
- $s_0 \in S$ is the starting state.
- $\delta \subseteq S \times \Sigma \times S$ is a transition function.
- $F \subseteq S$ is the set of accepted (final) state.

An input word $w \in \Sigma^*$ is an accepting word if there exists $f \in F$ such that $s_0 \vdash^* f$. The formal language L recognized by the automaton is the set of all accepting words, called the recognizable language of the acceptor.

iv Transducer

A transducer is an automaton with output and can be defined as a five-tuple

$$M = (\Sigma, \Gamma, S, s_0, \delta),$$

where

- Σ is an input alphabet.
- Γ is an output alphabet.
- S is a set of all states.
- $s_0 \in S$ is the starting state.
- $\delta \subseteq S \times \Sigma \times S \times \Gamma$ is a transition function with output, which are four-tuples of current state, input character, next state, and (next) output, in which each element is denoted as

  $$(s, \sigma) \to (t, x) \quad \text{or} \quad \delta(s, \sigma) = (t, x)$$

  for $(s, \sigma, t, x) \in \delta$.

  A general transition function with output and a general transducer is called nondeterministic. If δ satisfies the constraint that

  $$\forall (s, \sigma) \in S \times \Sigma : (s, \sigma, t, x) \in \delta \wedge (s, \sigma, u, y) \in \delta \implies t = u \wedge x = y,$$

the transition function with output and the transducer is called deterministic. A deterministic transition function with output can also be written as a two partial function

$$\delta : S \times \Sigma \to S$$

mapping pairs of current state and input character to next states and called (deterministic) transition function, and

$$\lambda : S \times \Sigma \to \Gamma$$

mapping pairs of current state and input to output characters and called (deterministic) next-output function or output function.

Given an input word $w = a_1 a_2 \dots a_n \in \Sigma^*$, for the ith step of the computation (also called the ith transition), for all $(s_{i-1}, a_i, s_i, x_i) \in \delta$, the transduce may transition from s_{i-1} to s_i and outputs x_i, denoted as $s_{i-1} \vdash s_i$ or $(s_{i-1}, w_i, \gamma_{i-1}) \vdash (s_i, w_{i+1}, \gamma_i)$, where $w_i = a_i a_{i+1} \dots a_n$, $w_{n+1} = \varepsilon$, $\gamma_i = x_1 x_2 \dots x_i$, and $\gamma_0 = \varepsilon$. s_n is called the final state.

v Extended transition function with output

The transition function with output $\delta \subseteq S \times \Sigma \times S \times \Gamma$ can be extended inductively into $\bar{\delta} \subseteq S \times \Sigma^* \times S \times \Gamma^*$, which are four-tuples of current state, remaining input word, final state, and output word since this step, with, for the empty string ε,

$$\{(s, \varepsilon, t, \gamma) \in \bar{\delta}\} = \{(s, \varepsilon, s, \varepsilon) \mid s \in S\},$$

and for any fixed string $wa \in \Sigma^*$ where $a \in \Sigma$ is the last symbol and $w \in \Sigma^*$ is the (possibly empty) rest of the string, for all string $\gamma x \in \Gamma^*$ where $x \in \Gamma$ is the last symbol and $\gamma \in \Gamma*$ is the (possibly empty) rest of the string,

$$(s, wa, t, \gamma x) \in \bar{\delta} \iff \exists (s, w, q, \gamma) \in \bar{\delta} \text{ s.t. } (q, a, t, x) \in \delta.$$

vi Finite-state machine (FSM), finite-state automaton (FSA), finite automaton, or state machine

A finite-state machine can be defined as a five-tuple

$$M = (\Sigma, S, s_0, \delta, F),$$

where

- Σ is a finite input alphabet.
- S is a finite non-empty set of all states.
- $s_0 \in S$ is the starting state.
- $\delta \subseteq S \times \Sigma \times S$ is a transition function.
- $F \subseteq S$ is the set of accepted final states.

A deterministic finite-state automaton is abbreviated to DFS, DFA, DFSA, or DFSM. A nondeterministic finite-state automaton is abbreviated to NFS, NFA, NFSA, or NFSM.

Theorem. The collection of all recognizable languages of all finite-state machine is the collection of all regular languages.

vii Finite-state transducer (FST)

A finite-state transducer is a transducer that is also a FSA and can be defined as a six-tuple

$$M = (\Sigma, \Gamma, S, s_0, \delta, F),$$

where

- Σ is a finite input alphabet.
- Γ is an output alphabet.
- S is a finite non-empty set of all states.
- $s_0 \in S$ is the starting state.
- $\delta \subseteq S \times \Sigma \times S \times \Gamma$ is a transition function with output.
- $F \subseteq S$ is the set of accepted final states.

A deterministic finite-state transducer is abbreviated to DFST. A nondeterministic finite-state automaton is abbreviated to NDFST or NFST.

viii Mealy machine

A Mealy machine is a deterministic FSM without the set of accepted final states (since all final states are accepted) and can be defined as a six-tuple

$$M = (\Sigma, \Gamma, S, s_0, \delta, G),$$

where

- Σ is a finite input alphabet.
- Γ is a finite output alphabet.
- S is a finite non-empty set of all states.
- $s_0 \in S$ is the starting state.
- $\delta : S \times \Sigma \to S$ is a deterministic transition function.
- λ is a deterministic next-output function.

ix Moore machine

A Moore machine is a Mealy machine whose output only depends on present state and not on input and can be defined as a six-tuple

$$M = (\Sigma, \Gamma, S, s_0, \delta, G),$$

where

- Σ is a finite input alphabet.
- Γ is a finite output alphabet.

- S is a finite non-empty set of all states.
- $s_0 \in S$ is the starting state.
- $\delta : S \times \Sigma \to S$ is a deterministic transition function.
- λ is a deterministic next-output function, which is a partial function

  $$\lambda : S \to \Gamma$$

  mapping current states to output characters.

x Pushdown automaton (PDA)

A pushdown automaton accepting by final state can be defined as a seven-tuple

$$M = (\Sigma, \Gamma, S, s_0, \delta, Z, F),$$

and a pushdown automaton accepting by empty stack can be defined as a six-tuple

$$M = (\Sigma, \Gamma, S, s_0, \delta, Z),$$

where

- Σ is a finite input alphabet.
- Γ is a finite stack alphabet.
- S is a finite non-empty set of all states.
- $s_0 \in S$ is the starting state.
- $\delta \subseteq S \times (\Sigma \cup \{\varepsilon\}) \times \Gamma \times S \times \Gamma^*$ is a transition function, which are five-tuples of current state, input character or empty string, current stack symbol to pop, next state, and stack word to push, in which each element is denoted as

  $$(s, \sigma, x) \to (t, \gamma) \quad \text{or} \quad \delta(s, \sigma, x) = (t, \gamma)$$

  for $(s, \sigma, x, t, \gamma) \in \delta$.

  A general transition function and a general PDA is called nondeterministic. If δ satisfies the constraints that

  $$\forall (s, \sigma, x) \in S \times (\Sigma \cup \{\varepsilon\}) \times \Gamma : (s, \sigma, x, t, \gamma) \in \delta \land (s, \sigma, x, u, \rho) \in \delta \implies t = u \land \gamma = \rho,$$

  and that

  $$\forall (s, x) \in S \times \Gamma : (\exists (t, \gamma) \in S \times \Gamma^* : (s, \varepsilon, x, t, \gamma) \in \delta) \implies (\forall (\sigma, t, \gamma) \in \Sigma \times S \times \Gamma^* : (s, \sigma, x, t, \gamma) \notin \delta)$$

  the transition function and the PDA is called deterministic, where the second constraint is called the no ε/normal conflict. A deterministic transition function can also be written as a partial function

  $$\delta : S \times (\Sigma \cup \{\varepsilon\}) \times \Gamma \to S \times \Gamma^*$$

mapping pairs of current state, input character or empty string, and current stack symbol to pop, to pairs of next state and stack word to push, with the no ε/normal conflict constraint that

$$\forall (s, x) \in S \times \Gamma : (s, \varepsilon, x) \in D_\delta \implies (\forall \sigma \in \Sigma : (s, \sigma, x) \notin D_\delta.$$

- $Z \in \Gamma$ is the initial stack symbol,
- $F \subseteq S$ is the set of accepted final states.

The stack of a PDA is a string over Γ. When the PDA pops a symbol from the stack, the leftmost symbol is removed from the stack, that is, the stack transitions from $x\gamma$ to γ where $x \in \Gamma$ and $\gamma \in \Gamma^*$. When the PDA pushes a word $\rho \in \Gamma^*$ to the stack, ρ is prepended to the left of the sequence, that is, the stack transitions from γ to $\rho\gamma$ where $\gamma \in \Gamma^*$.

Given an input word $w = a_1 a_2 \dots a_n \in \Sigma^*$ and a starting state $s_0 \in S$ of a PDA, for each ith step of the computation, a PDA reads an input character a, when which the transition is called a normal transition, or reads an empty string, when which the transition is called a ε-transition, pops the leftmost symbol $x \in \Gamma$ from the stack, transitions its state according to δ from s_{i-1} to s_i, and pushes a stack word $y \in \Gamma^*$ to the stack according to δ, denoted, with the remaining input word before the transition being av and the stack before the transition being $x\alpha$, for normal transition as $(s_{i-1}, av, x\alpha) \vdash (s_i, v, y\alpha)$ and for ε-transition as $(s_{i-1}, av, x\alpha) \vdash (s_i, av, y\alpha)$.

For a pushdown automaton accepting by final state, an input word $w \in \Sigma^*$ is an accepting word if $(s_0, w, Z) \vdash^* (f, \varepsilon, \gamma)$ for some $f \in F$ and $\gamma \in \Gamma^*$. For each pushdown automaton M accepting by final state, the set of all accepting words is denoted as $L(M)$.

For a pushdown automaton accepting by empty stack, an input word $w \in \Sigma^*$ is an accepting word if $(s_0, w, Z) \vdash^* (s, \varepsilon, \varepsilon)$ for some $s \in S$. For each pushdown automaton M accepting by empty stack, the set of all accepting words is denoted as $N(M)$.

A deterministic PDA is abbreviated to DPDA. A nondeterministic PDA is abbreviated to NPDA.

Theorem. For each pushdown automaton M accepting by final state, there exists a pushdown automaton M' accepting by empty stack such that $L(M) = N(M')$. For each pushdown automaton M accepting by empty stack, there exists a pushdown automaton M' accepting by final state such that $N(M) = L(M')$.

Theorem. The collection of all recognizable languages of all pushdown automata is the collection of all context-free languages.

xi Turing machine (TM)

A Turing machine can be defined as a seven-tuple

$$M = (\Sigma, b, \Gamma, S, s_0, \delta, F)$$

where

- Σ is a finite input alphabet.

- $b \notin \Sigma$ is the blank symbol.
- $\Gamma \supseteq \Sigma \cup \{b\}$ is a finite tape alphabet.
- S is a finite non-empty set of all states.
- $s_0 \in S$ is the starting state.
- $\delta \subseteq S \times \Gamma \times S \times \Gamma \times \{L, S, R\}$ is a transition function, which are five-tuples of current state, current tape symbol at head, next state, tape symbol to overwrite at head, and head movement, where L means moving head one cell left, S means not moving head, R means moving head one cell right, in which each element is denoted as

  $$(s, x) \to (t, y, c) \quad \text{or} \quad \delta(s, x) = (t, y, c)$$

  for $(s, x, t, y, c) \in \delta$.

  A general transition function and a general Turing machine is called nondeterministic. If δ satisfies the constraint that

  $$\forall (s, x) \in S \times \Gamma : (s, x, t, y, c) \in \delta \land (s, x, u, z, d) \in \delta \implies t = u \land y = z \land c = d,$$

  the transition function and the Turing machine is called deterministic. A deterministic transition function can also be written as a partial function

  $$\delta : S \times \Gamma \to S \times \Gamma \times \{L, S, R\}$$

  mapping pairs of current state and current tape symbol at head to triples of next state, tape symbol to overwrite at head, and head movement.
- $F \subseteq S$ is the set of accepted final states.

Given input word $w = a_1 a_2 \dots a_n \in \Sigma^*$, the tape of a TM is an infinite sequence initially loaded with w in a contiguous block from left to right and loaded with b for the rest of the cells. Let the cell loaded with a_i be of index i. The initial head is the cell of index 1. For one-way infinite tape, the cell of index 1 is the leftmost cell, and moving head left at that cell is defined as staying at that cell. For two-way infinite tape, there are infinite cells to both the left and right to the cell of index 1.

For each ith step of the computation, the TM reads the tape symbol at the current head, transitions its state according to δ from s_{i-1} to s_i, overwrite the symbol at the current head according to δ, and move the head one cell left or right or not move the head according to δ.

If current state s and current tape symbol at head x is such that $\forall (t, y, c) \in S \times \Gamma \times \{L, S, R\} : (s, x, t, y, c) \notin \delta$, the TM is said to halt and s is called the final state. If the final state of a TM is in F, the halt is an accepting halt and the input word is an accepting word; otherwise, the halt is a rejecting halt, the final state is a rejecting state, and the input word is a rejecting word.

A deterministic TM is abbreviated to DTM. A nondeterministic TM is abbreviated to NTM.

Theorem. The collection of all recognizable languages of all Turing machine is the collection of all recursively enumerable languages.

xii Linear bounded automaton (LBA)

A linear bounded automaton is a TM with the constraint that its head may only move in a region containing the whole input word of a length linear to the length of input word and can be defined as a eight-tuple

$$M = (\Sigma, l, r, \Gamma, S, s_0, F, \delta)$$

where

- Σ is a finite input alphabet.
- $l, r \notin \Sigma$ is the left and right end marker.
- $\Gamma \supseteq \Sigma \cup \{l, r\}$ is a finite tape alphabet.
- S is a finite non-empty set of all states.
- $s_0 \in S$ is the starting state.
- $\delta \subseteq S \times \Gamma \times S \times (\Gamma \setminus \{l, r\}) \times \{L, S, R\}$ is a transition function, which are five-tuples of current state, current tape symbol at head, next state, tape symbol to overwrite at head, and head movement, where L means moving head one cell left, S means not moving head, R means moving head one cell right, with the constraints that

  $$(s, l, t, y, c) \in \delta \implies c \neq L,$$

  and

  $$(s, r, t, y, c) \in \delta \implies c \neq R,$$

  in which each element is denoted as

  $$(s, x) \to (t, y, c) \quad \text{or} \quad \delta(s, x) = (t, y, c)$$

  for $(s, x, t, y, c) \in \delta$.

  A general transition function and a general LBA is called nondeterministic. If δ satisfies the constraint that

  $$\forall (s, x) \in S \times \Gamma : \ (s, x, t, y, c) \in \delta \land (s, x, u, z, d) \in \delta \implies t = u \land y = z \land c = d,$$

  the transition function and the LBA is called deterministic. A deterministic transition function can also be written as a partial function

  $$\delta : \ S \times \Gamma \to S \times (\Gamma \setminus \{l, r\}) \times \{L, S, R\}$$

  mapping pairs of current state and current tape symbol at head to triples of next state, tape symbol to overwrite at head, and head movement, with the constraint that

  $$\delta(s, l) = (t, y, c) \implies y = l \land c \neq L,$$

  and

  $$\delta(s, r) = (t, y, c) \implies y = r \land c \neq R.$$

- $F \subseteq S$ is the set of accepted final states.

Given input word $w = a_1 a_2 \dots a_n \in \Sigma^*$, the tape of a LBA is initially loaded with lwr in a contiguous block from left to right. The initial head is the cell loaded with a_1.

For each ith step of the computation, the LBA reads the tape symbol at the current head, transitions its state according to δ from s_{i-1} to s_i, overwrite the symbol at the current head according to δ, and move the head one cell left or right or not move the head according to δ.

If current state s and current tape symbol at head x is such that $\forall (t, y, c) \in S \times \Gamma \times \{L, S, R\} : (s, x, t, y, c) \notin \delta$, the LBA is said to halt and s is called the final state. If the final state of a LBA is in F, the halt is an accepting halt and the input word is an accepting word; otherwise, the halt is a rejecting halt, the final state is a rejecting state, and the input word is a rejecting word.

A deterministic LBA is abbreviated to DLBA. A nondeterministic LBA is abbreviated to NLBA.

Theorem. The collection of all recognizable languages of all linear bounded automaton is the collection of all context-sensitive languages.

xiii State-transition table for FSM and FST

A one-dimensional state-transition table with three or four (for FST) that lists the corresponding next states and outputs (for FST) in the third and fourth columns for all allowed combinations of input character in the first column and present state in the second column in rows, resembling a truth table.

A two-dimensional state-transition table consists of rows of all present states and columns of all input characters and specifies the corresponding next states (and outputs for FST) in each cell, where the cell of not allowed combinations of input and current state is typically filled with -.

The states may be represented in meaningless symbols or real meaning.

xiv State diagram or state graph for Moore machine

A state diagram (also called a state graph) for a Moore machine is a directed graph with

- Each vertex being a state, normally represented by circles with states in the top half and output at the state in the bottom half, i.e., [state]/[output character].
- Each directed edge (arc) being a transition, normally drawn as an arrow from present state to next state with the input character read for such transition to happen on the edge.
- The starting state is usually represented by an arrow with no origin pointing to the state, a subscript 0 in the symbol of the state (often s_0 or S_0), or not shown.

xv State diagram or state graph for Mealy machine

A state diagram (also called a state graph) for a Mealy machine is a directed graph with

- Each vertex being a state, normally represented by circles with states in it.
- Each directed edge (arc) being a transition, normally drawn as an arrow from present state to next state with the input character read for such transition to happen and output character of the transition on the edge separated by /, i.e., [input character]/[output character].
- The starting state is usually represented by an arrow with no origin pointing to the state, a subscript 0 in the symbol of the state (often s_0 or S_0), or not shown.

III Computability Theory

i Computably enumerable (c.e.), recursively enumerable (r.e.), semidecidable, partially decidable, listable, provable or Turing-recognizable set

A set S of words is computably enumerable (c.e.), recursively enumerable (r.e.), semidecidable, partially decidable, listable, provable or Turing-recognizable is there exists a Turin machine whose set of accepting word is S.